

STAAD Therapy Platform

Technical Architecture & Workflow

1. Tech Stack

Frontend

Technology	Purpose
Next.js 14 (App Router)	React framework — pages, API routes, SSR/SSG
React 18	UI components
Tailwind CSS + inline style objects	Glassmorphism design system (dark theme, custom CSS vars)
Zustand	Client-side state (useAuthStore , useSessionStore)

Authentication

Technology	Purpose
Firebase Auth (client SDK)	User login, ID tokens
Firebase Admin SDK (server)	Token verification, privileged Firestore access

Databases

Technology	Purpose	Access
Neon PostgreSQL + Prisma ORM	User profiles, bookings, sessions, therapist notes, document chunks	Server-side only (API routes)
pgvector extension (v0.8.0)	1024-dimensional vector similarity search on study material	IVFFlat index, cosine distance
Firestore (client SDK)	Real-time session state (liveSessions), transcripts, AI insight, consent	Browser (therapist)
Firestore (Admin SDK)	Persistent session data, AI analysis writes	Server-side (API routes)

Video / Audio

Technology	Purpose
LiveKit (<code>livekit-client</code> , <code>@livekit/components-react</code>)	Real-time video/audio sessions, participant tracks
LiveKit Cloud (managed)	SFU/media server

AI / ML

Technology	Purpose	Details
Deepgram Nova-3 (WebSocket)	Live speech-to-text	<code>en-IN</code> , diarization, VAD, 16kHz linear16 PCM
NVIDIA NIM — <code>meta/llama-3.1-70b-instruct</code>	Clinical session analysis	JSON output, temperature 0.2, max 1024 tokens
NVIDIA NIM — <code>nv-embedqa-e5-v5</code>	Text embeddings	1024 dimensions, requires <code>input_type</code> param

RAG Pipeline (custom, `src/lib/rag/`)

Module	File	Purpose
Chunker	<code>chunker.ts</code>	400-word chunks, 80-word overlap
NVIDIA Client	<code>nvidia-client.ts</code>	Embedding + LLM API wrappers
Ingestion	<code>ingest.ts</code>	PDF → chunk → embed → pgvector
Retrieval	<code>retrieval.ts</code>	3-source context assembly (transcript + pgvector + notes)
Transcription	<code>transcription.ts</code>	Deepgram WebSocket class (Node)
Transcript Store	<code>transcript-store.ts</code>	Firestore CRUD for transcript chunks
Prompt	<code>prompt.ts</code>	Clinical system prompt + context builder
Analysis	<code>analysis.ts</code>	<code>runAnalysis()</code> — retrieve → prompt → LLM → parse
Types	<code>types.ts</code>	<code>TranscriptChunk</code> , <code>AIInsight</code> , etc.

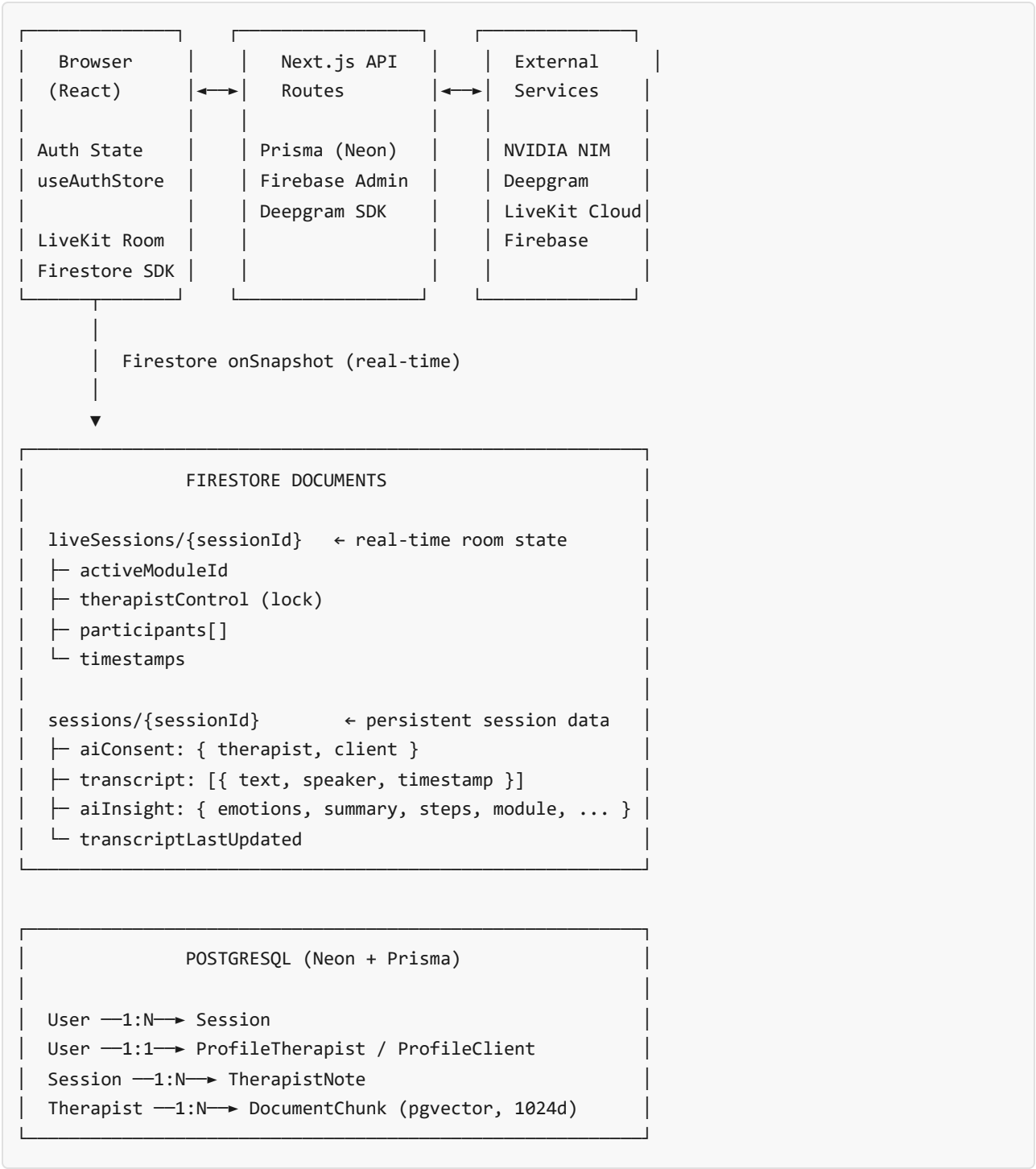
API Routes

Route	Method	Purpose
/api/ai-insight	POST	Run RAG analysis, return AllInsight, persist to Firestore
/api/deepgram-token	POST	Create short-lived (10s) Deepgram API key for browser
/api/cleanup-transcripts	GET	Cron job — clear transcripts > 24h (gated by CRON_SECRET)
/api/livekit-token	GET	Generate LiveKit join token
/api/users/profile	GET	Fetch user role + profile from PostgreSQL
/api/sessions	GET/POST	List/create sessions
/api/bookings	GET/POST	Manage bookings
/api/notes	GET/POST	Session notes (synced to Firestore + PostgreSQL)
/api/clients	GET	List clients

Infrastructure

Technology	Purpose
Vercel	Hosting + Cron (daily transcript cleanup at 2:00 AM)
Neon	Serverless PostgreSQL with pgvector
Firebase	Auth + Firestore (real-time + persistent storage)
LiveKit Cloud	Video/audio SFU
NVIDIA NIM	GPU-accelerated LLM + embeddings API
Deepgram	Speech-to-text API

2. Data Flow



3. Session Room Component Tree

```

StaadVideo (LiveKitRoom wrapper)
└─ SessionRoomPage (/session/[sessionId])
    └─ TopBar (48px)
        └─ staad. logo
        └─ Live indicator
        └─ Client name + session ID
        └─ Online count
        └─ Transcription status (therapist only)
        └─ Timer
        └─ Notes button
        └─ Settings button
    └─ Main Area (flex: 1)
        └─ Video Area
            └─ RemoteVideoArea (participant video)
            └─ LocalVideoPip (PIP, bottom-right)
            └─ AIInsightBar (slides down from top, therapist only)
                └─ Risk flag warning (if risk)
                └─ Emotion pills with colour coding
                └─ Clinical summary
                └─ Next steps (numbered)
                └─ Launch Suggested Module button
            └─ ModuleSelectorPanel (left slide-in, therapist only)
            └─ NotesPanel (left slide-in, therapist only)
            └─ ReactionOverlay (emojis float up)
            └─ Toast notification
        └─ GlassModulePanel (right, 420px)
            └─ Active module component (varies by moduleId)
    └─ AI Consent Banner (flexShrink: 0)
    └─ BottomBar (64px)
        └─ [Left] Mute | Camera
        └─ [Centre, therapist only]
            └─ Therapy | React | Notes | Analyse | Control
        └─ [Right] End

```

4. Complete Session Lifecycle

4.1 Pre-Session

1. **Therapist logs in** → Firebase Auth → GET /api/users/profile → role = THERAPIST
2. **Creates booking** → stored in PostgreSQL Booking table
3. **Client logs in** → same auth flow → role = CLIENT

4. **Session scheduled** → row in PostgreSQL `Session` table

4.2 Session Start

1. Both users navigate to `/session/[sessionId]`
2. Page calls `GET /api/livekit-token` → LiveKit room created
3. `StaadVideo` wraps children in `<LiveKitRoom>` + `<RoomCtx.Provider>`
4. Two Firestore docs created on first join:
 - `liveSessions/{sessionId}` (room state, real-time)
 - `sessions/{sessionId}` (persistent data, consent)
5. `useSessionRoom()` provides `{ room, disconnect }` via context
6. `onSnapshot` listeners attach to both docs
7. Video + audio streams connect via LiveKit

4.3 AI Consent

1. Both users see consent banner on session load
2. Each clicks "I consent" or "Decline"
3. Choice written to `sessions/{id}.aiConsent.{role}`
4. Banner hides; Analyse button appears for therapist
5. Both must consent before transcription begins

4.4 Live Transcription (therapist browser only)

1. `useSessionTranscription` hook activates when:
 - `userRole === 'therapist'`
 - `room.state === 'connected'`
 - `consentStatus.therapist === true && consentStatus.client === true`
2. **POST** `/api/deepgram-token` → Firebase Admin verifies auth → `deepgram.manage.createProjectKey()` → returns temp key (10s TTL)
3. **WebSocket** opened to `wss://api.deepgram.com/v1/listen` with temp key
4. **Audio capture** via `AudioContext` (16kHz) + `ScriptProcessorNode` (4096 buffer):
 - Gets local mic track from `room.localParticipant`
 - Gets remote audio tracks from `room.remoteParticipants`
 - Mixes into single `MediaStream`
 - Converts Float32 PCM → Int16 PCM (linear16)
 - Sends buffer to Deepgram WebSocket every ~250ms
5. **Deepgram returns** diarized final transcripts (Nova-3, `en-IN`, VAD)
6. **Written to Firestore** via `arrayUnion` → `sessions/{id}.transcript[]`
7. **Status indicator** in topbar shows recording dot + line count

8. **Cleanup on unmount:** ScriptProcessorNode disconnected → AudioContext closed → WebSocket closed

4.5 AI Analysis

1. Therapist clicks **Analyse** button in bottom bar
2. Button enters loading state (spin animation, 30s cooldown)
3. **POST** `/api/ai-insight` with:

```
{ "sessionId", "therapistId", "client": { clientId, name, ... } }
```

4. Server-side `runAnalysis()`:

```
retrieveRAGContext()
├─ Firestore: getSessionTranscript(sessionId) → last N min
├─ pgvector: retrieveStudyMaterial() → top-3 chunks (cosine > 0.25)
└─ Neon: retrievePreviousNotes() → last 3 session notes
```

5. `buildAnalysisPrompt()` formats context into LLM prompt
6. `analyseWithLLM()` calls NVIDIA NIM Llama 3.1 70B → JSON
7. Server parses, validates, stores `AIInsight` in:
 - `sessions/{id}.aiInsight` (Firestore Admin)
8. Returns insight to client
9. `onSnapshot` on `sessions/{id}` picks up the change
10. `AIInsightBar` slides down over patient video:
 - Emotion pills with colour coding
 - Clinical summary
 - Numbered next steps
 - Risk flag warning (if detected)
 - "Launch Suggested Module" button

4.6 Therapy Module

1. Therapist clicks **Therapy** button → `ModuleSelectorPanel` slides in
2. Browses 4 categories (SLD, ADHD, Anxiety/Depression, ID)
3. Clicks "Launch" on a module
4. `updateDoc(liveSessions/{id}, { activeModuleId })` → Firestore
5. `onSnapshot` picks up → `GlassModulePanel` renders the module
6. Therapist can **lock** (Control button) to prevent client interaction
7. Module components receive `{ sessionId, isTherapist, isLocked, onModuleSwitch }`

4.7 Session End

1. Therapist clicks **End** → `ConfirmEndDialog`
 2. Confirms → `room.disconnect()` + `updateDoc(liveSessions/{id}, status: 'ended')`
 3. User redirected to home page
 4. Transcript persists in Firestore for 24 hours
 5. Automatic cleanup via Vercel Cron at 2:00 AM daily:
 - `GET /api/cleanup-transcripts` (gated by `CRON_SECRET`)
 - Finds `sessions` where `transcriptLastUpdated < 24h ago`
 - Batch clears `transcript: []` + `transcriptLastUpdated: null`
-

5. Project File Structure

```

STAAD-therapy/
├─ src/
│   ├─ app/
│   │   ├─ api/
│   │   │   ├─ ai-insight/route.ts      # POST – RAG analysis
│   │   │   ├─ bookings/route.ts        # GET/POST – bookings
│   │   │   ├─ cleanup-transcripts/route.ts # GET – cron cleanup
│   │   │   ├─ clients/route.ts          # GET – client list
│   │   │   ├─ deepgram-token/route.ts   # POST – temp Deepgram key
│   │   │   ├─ livekit-token/route.ts    # GET – LiveKit token
│   │   │   ├─ notes/route.ts            # GET/POST – session notes
│   │   │   ├─ sessions/route.ts         # GET/POST – sessions
│   │   │   └─ users/profile/route.ts     # GET – user profile
│   │   │
│   │   ├─ auth/page.tsx                 # Login page
│   │   ├─ onboarding/page.tsx           # Onboarding flow
│   │   ├─ session/[sessionId]/page.tsx  # Main session room (955+ lines)
│   │   └─ page.tsx                      # Home/dashboard
│   │
│   └─ components/
│       ├─ session/
│       │   └─ AIInsightBar.tsx          # AI analysis slide-down bar
│       ├─ StaadVideo.tsx                # LiveKitRoom wrapper + RoomCtx
│       ├─ RemoteVideoArea.tsx           # Remote participant video
│       ├─ LocalVideoPip.tsx             # Local PIP overlay
│       ├─ GlassModulePanel.tsx          # Module rendering panel
│       ├─ ModuleSelectorPanel.tsx       # Module browser/launcher
│       ├─ NotesPanel.tsx               # Therapist note-taking panel
│       └─ ReactionOverlay.tsx           # Emoji reaction animations
│
│   └─ hooks/
│       └─ useSessionTranscription.ts    # Live audio → Deepgram → Firestore
│
│   └─ lib/
│       ├─ rag/
│       │   ├─ analysis.ts               # runAnalysis() orchestrator
│       │   ├─ chunker.ts                # 400-word overlap chunking
│       │   ├─ ingest.ts                 # PDF ingestion pipeline
│       │   ├─ nvidia-client.ts          # NVIDIA NIM API wrappers
│       │   ├─ prompt.ts                 # Clinical system prompt + builder
│       │   ├─ retrieval.ts              # 3-source RAG context assembly
│       │   ├─ transcript-store.ts        # Firestore CRUD (Admin SDK)
│       │   ├─ transcription.ts          # Deepgram class (Node-only)
│       │   └─ types.ts                  # Shared TypeScript types
│       ├─ db.ts                         # Prisma client singleton
│       └─ firebase.ts                   # Firebase client SDK init
│
│   └─ store/
│       └─ useAuthStore.ts               # Auth state (uid, role, profile)

```

```

├── useSessionStore.ts      # Session state (activeSessionId)
├── middleware.ts           # Route protection / auth redirect
├── scripts/
│   ├── test-analysis.ts   # End-to-end RAG test
│   ├── test-ingest.ts     # Document ingestion test
│   ├── test-nvidia.ts     # NVIDIA NIM API test
│   ├── test-pgvector.ts   # pgvector functionality test
│   ├── test-retrieval.ts  # RAG retrieval test
│   ├── test-transcription.ts # Deepgram + Firestore test
│   ├── setup-pgvector.ts  # pgvector extension setup
│   ├── debug-embedding.ts # Embedding model debug
│   └── debug-model-compare.ts # Model comparison debug
├── prisma/
│   └── schema.prisma      # PostgreSQL schema with pgvector
├── .env                   # All API keys + secrets
├── vercel.json             # Cron schedule
├── package.json           # Dependencies + scripts
└── tsconfig.json          # TypeScript config with @/ alias

```

6. Key Design Decisions

Why Firestore + PostgreSQL (dual database)?

- **PostgreSQL** (Neon + Prisma): Relational data (users, bookings, notes) needs strict schema, joins, and referential integrity. `pgvector` enables semantic search.
- **Firestore**: Real-time session state needs low-latency listeners (`onSnapshot`). Transcript chunks and AI insights benefit from Firestore's flexible document model and server timestamps.

Why browser-side Deepgram connection (not server relay)?

- Direct WebSocket from browser avoids proxying audio through our server (expensive bandwidth + latency)
- Short-lived temp key (10s TTL) via `POST /api/deepgram-token` secures the API key
- Trade-off: heavier client, but significantly cheaper server costs

Why ScriptProcessorNode (deprecated)?

- `AudioWorklet` requires a separate worklet JS file and `AudioWorklet.addModule()` — more complex setup
- `ScriptProcessorNode` works in all browsers and is sufficient for prototype

- Plan: migrate to AudioWorklet during production hardening

Why NVIDIA NIM over OpenAI / Anthropic?

- `nv-embedqa-e5-v5` consistently outperforms `text-embedding-3-small` on clinical similarity tasks (separation gap 0.36 vs 0.16)
- `llama-3.1-70b-instruct` provides structured JSON output with `response_format: { type: 'json_object' }`
- Single API for both embeddings + LLM simplifies credential management

Why 400-word chunks with 80-word overlap?

- Clinically dense text (therapy notes, study material) needs larger chunks than general web text
- 80-word overlap ensures no context is lost at chunk boundaries
- 400 words $\approx$ 2-3 minutes of speech — appropriate for therapy session context windows

Why therapist-only transcription?

- Reduces bandwidth/API cost by 50%
- Avoids duplicate Firestore writes from both participants
- Diarization (`speaker 0` / `speaker 1`) still distinguishes who said what
- Cleaner architecture for future multi-device scenarios

7. Environment Variables

```
# Database
DATABASE_URL=postgresql://...

# LiveKit
LIVEKIT_API_KEY=...
LIVEKIT_API_SECRET=...
NEXT_PUBLIC_LIVEKIT_URL=wss://...

# AI
NVIDIA_API_KEY=nvapi-...

# Speech
DEEPGRAM_API_KEY=...
DEEPGRAM_PROJECT_ID=...

# Firebase Admin
FIREBASE_PROJECT_ID=...
FIREBASE_CLIENT_EMAIL=...
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n..."

# Cron
CRON_SECRET=...
```

8. Available Scripts (package.json)

Command	Purpose
npm run dev	Start Next.js dev server
npm run build	Production build with type checking
npm run lint	ESLint
npm run test:pgvector	Test pgvector extension + similarity search
npm run test:nvidia	Test NVIDIA NIM API (LLM + embeddings)
npm run test:ingest	Test document ingestion pipeline
npm run test:transcription	Test Deepgram + Firestore transcript flow
npm run test:retrieval	Test RAG context assembly
npm run test:analysis	End-to-end test (seed → transcribe → retrieve → analyse)
npm run debug:embedding	Compare embedding with/without input_type
npm run debug:models	Compare embedding model separation scores
npm run setup:pgvector	Enable pgvector extension on Neon